

Documentazione dell'Architettura Software

Progetto: Music Playlist Manager (Gruppo 10)

Linguaggio e Tecnologie: Java, JavaFX (Interfaccia Grafica), JUnit (Unit Testing).

1. Architettura di Alto Livello (MVC + Service Layer)

L'applicazione è strutturata seguendo il pattern architetturale **MVC (Model-View-Controller)** **assieme ad un Service Layer**, garantendo un'elevata coesione dei componenti e un basso livello di accoppiamento globale. La separazione dei compiti segue regole di comunicazione precise:

- **View:** Rappresenta l'interfaccia utente grafica sviluppata tramite componenti JavaFX e file descrittivi FXML. Invia gli input e gli eventi intercettati al rispettivo Controller.
- **Controller:** Gestisce la logica di presentazione della schermata specifica. Riceve i comandi dalla View, aggiorna gli elementi grafici in tempo reale e interagisce sia con il Model che con il Service Layer.
- **Service:** Uno strato intermedio indipendente adibito alla logica di business di sistema e infrastrutturale (es. servizi globali di navigazione o riproduzione attiva). Dialoga con il Model e risponde alle richieste dei Controller.
- **Model:** Incapsula le entità core del dominio, memorizzando lo stato dei dati dell'applicazione direttamente in memoria.

2. Dettaglio del Service Layer

La logica dell'applicazione è orchestrata da componenti centralizzati implementati come **Singleton** per garantire un punto di accesso globale unico:

- **NavigationManager (Interfaccia Navigator):** Gestisce centralmente il flusso di navigazione tra le diverse schermate grafiche dell'applicazione (es. tramite i metodi `initRootLayout` e `navigateTo`), disaccoppiando i singoli controller dalla manipolazione diretta degli Stage.
- **ActivePlayerManager (Interfaccia PlayerManager):** Coordina lo stato di riproduzione musicale attivo a livello globale, fornendo metodi per avviare o terminare il player (`openPlayer`, `closePlayer`) e per interagire con i dati della traccia/playlist correntemente in esecuzione.

3. Design Pattern GoF Applicati

Il nucleo del sistema applica rigorosamente diversi Design Pattern della Gang of Four (GoF) per gestire la complessità dei requisiti legati alle modalità di riproduzione, alla gestione degli stati del lettore e alla funzionalità di Undo:

Design Pattern	Scopo nel Sistema	Classi / Componenti Coinvolti
Strategy	Permette di variare dinamicamente l'algoritmo di riproduzione delle playlist in base alla modalità scelta dall'utente.	ExecutionStrategy, Sequential, Shuffle, Loop
State	Modella il comportamento e la risposta del lettore musicale a seconda del suo stato dinamico interno (riproduzione o pausa).	PlayerState, Play, Pause
Command	Incapsula le operazioni di aggiunta e rimozione in oggetti comando dedicati, abilitando la gestione dello storico per l'Undo.	CommandInvoker, AbstractCommand, BasePlaylistCommands, BaseTrackCommands, AddPlaylistCommand, DeletePlaylistCommand, AddTrackCommand, RemoveTrackCommand
Observer	Garantisce il disaccoppiamento e la sincronizzazione reattiva tra il catalogo dei brani registrati e le playlist caricate in memoria.	SubjectTrackList, ObserverPlaylist, BaseSubjectTrackList, BaseObserverPlaylist

Design Pattern	Scopo nel Sistema	Classi / Componenti Coinvolti
Iterator	Fornisce un meccanismo uniforme per scorrere sequenzialmente i brani all'interno di una playlist, a prescindere dalla strategia di esecuzione attiva.	IterableCollection, AbstractIterator, Iterator, Playlist
Singleton	Garantisce che i gestori globali di sistema e le liste contenitore dei dati vivano in una sola istanza condivisa da tutta l'applicazione.	TrackList, PlaylistList, NavigationManager, ActivePlayerManager

4. Modello delle Classi Core

Il modello gestisce i dati strutturali essenziali relativi all'applicazione multimediale:

- **Track:** Modella il singolo brano musicale contenendone i dati identificativi (title, author, year) e i dati visuali e funzionali (favourite, explicit, newRelease, genre, seconds).
- **Playlist:** Caratterizzata da un attributo name, racchiude un insieme di tracce ordinate (ArrayList<Track>) e fornisce la logica interna per manipolare la lista in modo sicuro, esponendo anche un metodo di undo per supportare l'implementazione del pattern Command e metodi per l'aggiornamento delle tracce già esistenti.
- **TrackList e PlaylistList:** Classi contenitore regolate dal pattern Singleton che ospitano l'elenco complessivo dei brani e delle playlist create all'interno del ciclo di esecuzione.
- **Player:** Componente centrale incaricato di simulare e gestire la riproduzione delle canzoni all'interno dell'applicazione. Gestisce i componenti del modello dati e ne coordina le interazioni con i design pattern State, Strategy e Iterator:
 - **Attributi Interni:** Mantiene il riferimento alla playlist da cui è stato riprodotto il brano corrente, al brano in riproduzione (currentTrack di tipo Track), a un iteratore personalizzato (it: Iterator) e a un timer (timer: Timer) per tracciare il tempo trascorso (elapsedSeconds: int). Include lo stato d'esecuzione attuale (currentState di tipo PlayerState) e un consumer funzionale (onTimeTick: IntConsumer) delegato all'aggiornamento dell'interfaccia utente in tempo reale.
 - **Metodi Operativi:** Offre un'interfaccia completa per il controllo del flusso (startPlayback(), stopPlayback(), terminate()), la selezione direzionale dei brani guidata dalle strategie (getNext(), getPrevious()), la mutazione degli stati interni

(changeState()), e la scelta dinamica delle modalità di riproduzione (setStrategy(ExecutionStrategy)).